Mini Demo Car - Access Kuksa Data Broker – Tutorial

VSS Spec

Check

VSS Spec File (JSON)

VSS Spec File (CSV)

Prerequisites

```
pip install kuksa-client
```

---

1. Establishing a Connection (gRPC)

```
from kuksa_client.grpc import VSSClient

with VSSClient("192.168.56.6", 55555) as client:
    print("Connected to Kuksa Data Broker")
```

---

2. Multithreaded: Subscribe + Demo Sequence (`kuksa_sample.py`)

A shared `VSSClient` is used for both the subscriber thread and the control sequence.

```
import threading
import time
from kuksa_client.grpc import VSSClient, Datapoint

HOST = "192.168.56.6"
PORT = 55555

stop_event = threading.Event()


def subscriber_thread(client):
    """Listens to all changes below Vehicle.*"""
    print("[SUB] Subscribed to Vehicle.*")
    for updates in client.subscribe_current_values(["Vehicle"]):
        if stop_event.is_set():
            break
        for path, datapoint in updates.items():
```

```
            if datapoint and datapoint.value is not None:
                print(f"[SUB] {path} = {datapoint.value}")


def writeToKuksa(client, path, value):
    """Helper function to write a value"""
    client.set_current_values({
        path: Datapoint(value),
    })


def demo_sequence(client):
    """Example sequence: take over vehicle, start engine, accelerate, lights"""
    # Take control over Mini Demo Car (as Cloud)
    writeToKuksa(client, "Vehicle.RequestTakeOver", str([1, 120]))
    time.sleep(2)

    # Start engine
    writeToKuksa(client, "Vehicle.Powertrain.StartStop.StartControl", str([1,
120]))
    time.sleep(2)

    # Accelerate
    writeToKuksa(client, "Vehicle.Chassis.Accelerator.PedalPositionControl",
str([50, 120]))
    time.sleep(2)

    # Stop
    writeToKuksa(client, "Vehicle.Chassis.Accelerator.PedalPositionControl",
str([0, 120]))
    time.sleep(2)

    # Low beam on
    writeToKuksa(client, "Vehicle.Body.Lights.ExteriorLightControl", str([1, 5,
120]))
    time.sleep(2)

    # Low beam off
    writeToKuksa(client, "Vehicle.Body.Lights.ExteriorLightControl", str([0, 5,
120]))
    time.sleep(2)


if __name__ == "__main__":
    with VSSClient(HOST, PORT) as client:
        # Listen on everything below Vehicle.*
        sub = threading.Thread(target=subscriber_thread, args=(client,),
daemon=True)
        sub.start()

        demo_sequence(client)

        try:
            while True:
```

```
                time.sleep(0.1)
        except KeyboardInterrupt:
            print("\nStopping...")
            stop_event.set()
            sub.join(timeout=3)
```

Operating order

See the demo_sequence for some sample interactions.

1. Take control over Mini Demo Car with a ClientID
    1. Car: 100
    2. CIP: 110
    3. CLOUD: 120
2. Start engine
3. Steer, Accelerate, Brake, Turn Lights on or Off
4. (Optional) Stop Engine
5. (Optional) Give back control to ClientID 100 (just so the physical controls of the car work again, steering wheel, pedals, ...)

Signals

Signal Format (Array)

Arrays are passed as strings, e.g. `str([1, 120])` produces `"[1, 120]"`. The values have the following meaning (depending on the signal):

Signal	Array Format	Description
`Vehicle.RequestTakeOver`	`[1, ClientID]`	Request takeover
`Vehicle.Powertrain.StartStop.StartControl`	`[1, ClientID]`	Start engine
`Vehicle.Chassis.Accelerator.PedalPositionControl`	`[position, ClientID]`	Throttle position (0-100%)
`Vehicle.Body.Lights.ExteriorLightControl`	`[light_state, light_id, ClientID]`	Light control

Light Control

The actuator `Vehicle.Body.Lights.ExteriorLightControl` is used to change the state of a light.

For LowBeam e.g. `[1 (on), 5 (ID for Low Beam Left and Right simultaneously), 120 (ClientID)]`

To address only the left LowBeam: `[1 (on), 6 (ID for Low Beam Left), 120 (ClientID)]`

The STATE of a light can then be queried via its VSS signal name in the VSS tree: `Vehicle.Body.Lights.Beam.Low.Left`

for the state of the left low beam.

Light State (`light_state`)

Value	State
0	Off
1	On
2	Signaling
3	Adaptive

Light IDs (`light_id`)

ID	Light
0	All
1	Beam
2	HighBeam
3	HighBeamLeft
4	HighBeamRight
5	LowBeam
6	LowBeamLeft
7	LowBeamRight
8	Running
9	RunningFront
10	RunningFrontLeft
11	RunningFrontRight
12	RunningRear
13	RunningRearLeft
14	RunningRearRight
15	Backup
16	BackupLeft
17	BackupRight
18	Parking
19	ParkingFront
20	ParkingFrontLeft

ID	Light
21	ParkingFrontRight
22	ParkingRear
23	ParkingRearLeft
24	ParkingRearRight
25	Fog
26	FogFront
27	FogRear
28	LicensePlate
29	LicensePlateFront
30	LicensePlateRear
31	Brake
32	BrakeLeft
33	BrakeRight
34	Hazard
35	DirectionIndicator
36	DirectionIndicatorFront
37	DirectionIndicatorFrontLeft
38	DirectionIndicatorFrontRight
39	DirectionIndicatorRear
40	DirectionIndicatorRearLeft
41	DirectionIndicatorRearRight